

TECHNICAL REFERENCE

WeVo Pay

How everything works — for instructor questions

ASP.NET Core MVC · Entity Framework Core · SQL Server · Cookie Auth

Item	Detail
Project type	ASP.NET Core MVC web application
Purpose of this doc	Defend technical decisions and explain implementation
Related presentation	WeVo_Pay_Project_Presentation.pdf
Default admin	admin / Admin@123 (seeded on startup)

1. How to use this document

If the instructor asks HOW you built something, answer from the matching section:

- Architecture / layers / DI → Section 2–3
- Database / entities / migrations → Section 4
- Authentication & roles → Section 5
- Transfer business flow → Section 6
- Auto-cancel timer → Section 7
- Messages / referral / rates / settings → Section 8
- UI structure → Section 9
- Quick Q&A answers → Section 10

2. High-level architecture

WeVo Pay uses classic layered MVC:

- Views (Razor): HTML UI, forms, timers, layouts
- Controllers: receive HTTP, authorize, validate, call services, return views
- Services: business rules (fee, verify, complete, expire, messages)
- Data: ApplicationDbContext + EF Core maps models to SQL Server tables

Dependency Injection registers all services in Program.cs as Scoped. Background job is HostedService. Exchange rates use IHttpClientFactory + IMemoryCache.

Request path example (Create Transfer):

Browser POST /Transfer/Create

```
→ TransferController.Create(dto) [Authorize]
→ TransferService.CreateTransferAsync(userId, dto)
→ ApplicationDbContext.TransferRequests.Add(...) + SaveChanges
→ Redirect Success view
```

3. Project structure (what lives where)

Folder	Responsibility
Controllers/	HTTP endpoints, auth attributes, ViewBag
Services/ + Interfaces/	Business logic contracts + implementations
Models/	EF entities = database tables
DTOs/	Form/input/view models without EF navigation clutter
Data/	ApplicationDbContext, DbSeeder
Enums/	TransferStatus, UserRole, TransactionStatus
Views/	Razor pages per feature area
wwwroot/	CSS, JS, images (static files)
Migrations/	EF schema history

Main controllers

Controller	Role	Audience
------------	------	----------

AccountController	Register / Login / Logout	Public + auth
UserController	Dashboard, Profile, Settings	User
TransferController	Create, MyTransfers, Details, Success	User
AdminController	Queues: verify/complete/reject, stats	Admin
CompanyWalletController	Manage company wallets	Admin
SystemSettingController	Fee %, min/max amounts	Admin
MessageController	User chat + admin inbox	Both
RatesController	Currency + gold in EGP	User
TransactionController	Transaction listing/details	Admin

4. Database & domain model

ORM: Entity Framework Core. Provider: SQL Server. Connection: appsettings.json key ConString (case-insensitive).

Core tables

Entity	Key fields	Notes
User	UserName, Email, Phone, PasswordHash, Role	Role enum stored as string
CompanyWallet	WalletName, WalletNumber, IsActive	Platform mobile wallets
TransferRequest	Amount, Fee, Status, InstaPayAddress	Core business object
Transaction	Amount, ReferenceNumber, Status	Created on Complete
SystemSetting	FeePercentage, Min/Max amount	Global config
SupportMessage	UserId, SenderId, Body, IsFromAdmin	Support chat

Important relationships

- TransferRequest.UserId → User (many transfers per user)
- TransferRequest.CompanyWalletId → CompanyWallet
- Transaction 1:1 TransferRequest (created when completed)
- User.ReferredByUserId → User (optional self-reference)
- SupportMessage.UserId = conversation owner (customer)

Migrations that changed schema in this work

- AddSupportMessages → SupportMessages table
- AddUserReferral → Users.ReferredByUserId

The 1-hour auto-cancel does NOT change schema; it only updates Status to Cancelled.

5. Authentication & authorization

How login works

- UserService.LoginAsync finds user by email OR username (case-insensitive)
- PasswordHasher.VerifyHashedPassword checks password

- AccountController builds ClaimsIdentity with NameIdentifier, Name, Email, Role
- HttpContext.SignInAsync writes authentication cookie

Roles

- UserRole.User → user app (dashboard, transfers, rates, messages)
- UserRole.Admin → admin panel ([Authorize(Roles = "Admin")])

How to answer: Why cookies not JWT?

This is a server-rendered MVC app. Cookie auth is the standard, simple choice for browser sessions. JWT is more common for pure APIs/SPAs.

Admin seed

DbSeeder.SeedAdminAsync runs on startup: migrates DB and ensures admin exists with password Admin@123 (username admin).

6. Transfer engine (business + code)

Statuses

Status	Meaning	Who changes it
Pending	Created; waiting verification	User create / system start
Verified	Admin confirmed payment received	Admin Verify
Completed	Paid to InstaPay; Transaction row	Admin Complete
Rejected	Admin refused	Admin Reject
Cancelled	Timeout without verify (1 hour)	Background + on-read cancel

CreateTransferAsync steps

- Load user; fail if missing
- Load active CompanyWallet; fail if missing
- Load active SystemSetting; fail if missing
- Validate InstaPay address and amount vs Min/Max
- $\text{Fee} = \text{Round}(\text{amount} * \text{FeePercentage} / 100, 2)$
- $\text{TotalAmount} = \text{amount} + \text{fee}$
- Insert TransferRequest Status=Pending, CreatedAt=UtcNow

Verify / Complete / Reject

- Verify: only if Pending (after running expiry cancel); sets VerifiedAt + VerifiedByAdminId
- Complete: only if Verified and no Transaction yet; creates Transaction with reference WP-yyyyMMdd-XXXXXX
- Reject: from Pending or Verified; sets RejectedAt + RejectedByAdminId

Fee: is it constant?

No. Create page and CreateTransferAsync both read SystemSettings.FeePercentage. Changing fee in admin updates NEW transfers. Old transfers keep stored FeePercentage.

7. One-hour auto-cancel (timer)

Constant: `TransferService.PendingTimeout = TimeSpan.FromHours(1)`

Method: `CancelExpiredPendingTransfersAsync()`

```
cutoff =.UtcNow - 1 hour
find TransferRequests where Status==Pending AND CreatedAt <= cutoff
set Status = Cancelled; SaveChanges
```

Triggered by:

- `TransferExpiryBackgroundService` every ~1 minute (`IHostedService`)
- Also when listing/details/verify runs (so UI is not stale)

UI: countdown on Details and My Transfers; banners on Create/Success/Dashboard.

Why not only JavaScript timer?

JS countdown is display only. Real cancel must be server-side so it works even if the user closes the browser.

8. Other features (how they work)

Message Center

- `SupportMessage` rows; thread key = customer `UserId`
- User send: `IsFromAdmin=false`; Admin send: `IsFromAdmin=true`, `UserId=target`
- Opening chat marks the other side unread messages as `IsRead=true`
- `MessageController`: Roles User for Index/Send; Roles Admin for `AdminIndex/AdminConversation/AdminReply`

Referral

- `RegisterDto.WasReferred` + `ReferralCode`
- Parse username from plain name, `?ref=`, or `/ref/username`
- Store `Users.ReferredByUserId`
- Profile builds link: `/Account/Register?refCode=UserName`

User Settings

- `UpdateProfile`: unique email/phone excluding self; re-SignIn cookie
- `ChangePassword`: verify current hash, require different new password (min 6)

Exchange Rates

- `ExchangeRateService` calls `open.er-api.com` (USD base) → convert to EGP
- Gold: `gold-api.com` XAU price USD/oz → convert using USD/EGP
- Cached 30 minutes in `IMemoryCache`
- No DB tables for rates

9. UI / front-end technical notes

- Layouts: `_Layout.cshtml` (users/guests), `_AdminLayout.cshtml` (admin)
- Navbar branches: `FullWidth` (auth), `Authenticated Admin`, `Authenticated User`, `Guest`

- Styles mainly in wwwroot/css/wevopay.css and dashboard.css
- Client JS only for UX: fee calculator, password toggle, countdown timers, copy referral
- Real validation and security stay on the server

10. Instructor Q&A — ready answers

Q: Why MVC not Web API + React?

Course-style full-stack teaching target. Razor MVC keeps one project, clear Controllers/Views/Models separation, faster for academic delivery.

Q: Where is business logic?

In Services (e.g. TransferService), not in Views. Controllers orchestrate HTTP only.

Q: How do you prevent users seeing others' transfers?

GetUserTransferByIdAsync filters by transferId AND userId from claims. Admin endpoints require Admin role.

Q: How are passwords stored?

Never plain text. ASP.NET Core Identity PasswordHasher produces a salted hash in User.PasswordHash.

Q: How does the fee update when admin changes settings?

Create form loads FeePercentage from SystemSetting via ISystemSettingService. CreateTransferAsync recomputes fee from active settings at save time.

Q: How does auto-cancel work if the server restarts?

On next run, hosted service resumes and queries Pending older than 1 hour. Also lazy cancel on transfer reads.

Q: What if admin verifies after expiry?

VerifyTransferAsync first runs CancelExpiredPendingTransfersAsync. Expired pending becomes Cancelled, so verify returns false.

Q: How are messages secured?

User endpoints only for Role User and thread = current user id. Admin endpoints require Role Admin.

Q: Did every feature need a migration?

No. Messages and referral needed schema changes. UI polish, settings password change, timer cancel, and rates did not need new tables.

Q: What external APIs do you use?

open.er-api.com for FX; api.gold-api.com for gold spot. Results cached; failures degrade gracefully.

Q: How would you improve this for production?

- Move secrets to User Secrets / Azure Key Vault
- Add unit tests for TransferService status transitions

- Real-time messaging (SignalR)
- Payment confirmation proofs / upload receipts
- Audit log table for admin actions
- Stronger password policy and lockout

11. Key files cheat-sheet

Topic	Primary files
Startup / DI / seed	Program.cs, Data/DbSeeder.cs
DB mapping	Data/AppDbContext.cs, Models/*
Create/verify transfer	Services/TransferService.cs, Controllers/TransferController.cs, AdminController.cs
Timer	TransferService.CancelExpired..., TransferExpiryBackgroundService.cs
Auth	AccountController.cs, UserService.LoginAsync/RegisterAsync
Messages	MessageService.cs, MessageController.cs, Models/SupportMessage.cs
Rates	ExchangeRateService.cs, RatesController.cs
UI shell	Views/Shared/_Layout.cshtml, wwwroot/css/wevopay.css

12. Demo script (if asked to show)

- 1) Login as admin → Dashboard stats, pending list
- 2) Logout → Register/Login user → Create transfer → see 1h timer
- 3) Admin verify → complete → user sees Completed
- 4) Messages user ↔ admin
- 5) Rates page currencies + gold
- 6) Settings change password / profile

End of technical reference — use Section 10 for spoken answers.